

The Deadlock-Aware Ransomware Proof FUSE System.

Team Members:

1. Abhishek Maruti Karennavar - 241IT002
2. Sankalp S Kunthe - 241IT069
3. Santosh Gowda - 241IT070

Introduction:

In the modern world of computers, the problem faced at an individual level, as well as at the industrial level, from ransomware attacks is massive. The history of systems is a witness to several industry-level frauds, making the need for a section of the filesystem to overcome the problem and redefine the word security.

The deadlock-aware ransomware-proof fuse system is an implementation of a stackable Virtual Filesystem, aiming to bring protection from ransomware files, making the system foolproof while making sure that the system does not end up in a state of deadlock because of the implementation.

Project Overview:

The project implements a custom stackable Virtual Filesystem(VFS) using the FUSE library. The VFS makes sure that the ransomewares are detected in all the three levels of Architectures to neutralize them. The three-tier architecture spans as:

1. **Kernel/Fuse Tier:** Intercepts VFS I/O requests(sys_write), if suspicious, sends a signal to the os module to terminate the process (SIGKILL).
2. **IPC Tier (UNIX Sockets):** A deadlock aware IPC bridge.
3. **User Space ML tier:** an ML daemon that calculates the features, and categorises a file as benign or malignant.

Also, the system implements a program to oversee the processes and prevent circular waits so, the files do not end up in a state of deadlock.

Documentation and Explanation:

The explanation of the programme is divided into four modules, based on their role in the project. Namely the directories `src/fuse_fs`, `src/honeyfile_gen`, `src/kernel_ebpf`, `src/user_daemon` and the script file of `build_and_run.sh`

Caution before proceeding to implementation: Please use VMs to run the system, as it is not supposed to disturb the files in the Parent System. Link to Virtual box VM is [here](#). Use Ubuntu ISO image from the link [here](#).

Continuing the documentation,

1. `src/fuse_fs`: Contains a file namely `deadlock_aware_fuse.cpp`, which is a driver implemented in cpp. This program mounts the VFS and acts as a stackable passthrough to the physical backing store. This program handles POSIX file operations and the kernel-level process terminations.

Role of this file:

- This program translates the virtual path `‘/tmp/secure_mount/’` to physical disk location `‘/tmp/backing_store/’`.

```
void translate_path( char fpath[PATH_MAX], const char* path){
    strcpy(fpath, BACKING_STORE);
    strncat(fpath, path, PATH_MAX - strlen(BACKING_STORE) - 1);
}
```

- Function named `‘ransomware proof write’` acts as interceptor in write operations. The arguments are the path, buffer string, `size_t` datatype, an unsigned integer of offset (`off_t` type) and the `fuse_file_info`. It checks whether the target file is in Honeyfile list and if there in honeyfile list, the signal is sent to os to kill the process. The function adds the write to the buffer which will be sent to the Python daemon via sockets. If the daemon returns 1 as the verdict, the process is considered as malicious and returns `-EACCESS` to block I/O. The write operations is paused throughout the process.
- The function named `‘query_ml_daemon_with_timeout’` handles the IPC using the `AF_UNIX` (Address family to communicate between multiple processes in a same

machine) method. The `setsockopt` makes sure that no process gets into infinite circular wait, by using timeout of `SO_RCVTIMEO`, restricting the 500ms time to predict that it is deadlock, if nothing is received from `recv()`.

- Also once during the execution of main, the State initialization for whitelisted files and the honeytrap/tripwire files is done. The functions `load_daemon_pid` and `load_honeyfiles` will load the whitelisted files' PID and honeyfiles' path respectively from the physical disk to the kernel memory for zero latency lookup during active I/O operations.
2. *src/honeyfile_gen/*: This directory contains a file namely `generate_decoy.py` creates some decoy files with some attractive names for the ransomwares, and registers them for the C++ driver to monitor. Basically, the files generated by this act as baits for the Ransomware processes. As, these are the files, which are traps, OS processes will not be using them, if any process uses them, such processes are flagged as the malicious files by the driver module. Then, such processes are terminated. The script deliberately bypasses the virtual mount `'/tmp/secure_mount'` and stores at `'/tmp/backing_store'`. After creating these files, the virtual path is created for these files and saves this list to `'./tmp/backing_store/honeyfiles.txt'`.
 3. *src/user_daemon/*: This directory contains a file named `ml_daemon.py` which acts as an AI Inference Engine. The file listens on the `/tmp/ransomware_defense.sock` and evaluates file buffers interceptor by kernel.
 - At first, the daemon will write its PID in the `/tmp/ml_daemon.pid`. The C++ kernel uses the pid file to make an exception from the VFS interception. By whitelisting the daemon's PID, the kernel driver implementation, will not allow "Hold and wait" criteria to be true, thus making the kernel driver immediately pass the daemon's I/O requests straight to the disk ensuring stability.
 - It binds the AF_UNIX to `/tmp/ransomware_defense.sock`, to handle high-speed IPC, waiting continuously for 64KB file chunks sent by the FUSE driver.
 - When a buffer is received, the program calls for the `entropy_calc.py` calculating 6 statistical features in real-time. The features are stored as a numpy array feature vector.
 - The feature vector is passed on to the pre-trained Random Forest model (`rf_model.predict()`). The model, classifies binarily, 1- Malignant; 0 - Benign.

- The file send the verdict b"0" or b"1", allowing the process to be flagged and killed if it is b"1".
4. *src/ml_pipeline/* : This directory contains a program called `entropy_calc.py` and the sub directories called `datasets` and `models`. The `entropy_calc.py` will calculate the six features each of which represent a statistical measure on the sequence of bits. The measures are namely: shannon entropy, Chi-square, Monobit test, Poker Test, Cumulative Sums, MAC Metadata.
- The `ml_pipeline/datasets/generate_real_ransomware_data.py` helps in generation of dataset simulating ransomware behaviour. The program doesn't generate random sequence of bits. It generates data that mathematically reflects cryptographical payloads as modern ransomwares use AES-256 to encrypt the files. Creates an initialisation vector of 16 - bytes for every files. On encryption, the cipher is completely different for identical text sequences. Because AES requires data in strict 16-byte blocks, the script calculates and appends standard byte padding to the end of the original file data, mirroring how real-world ransomware prepares files for the cipher block. As the AES-256 takes input of size 128 bytes, the remainder bits or the chunk < 16 bytes will be padded with random sequence to make it 16 bytes long. Also, the length of sequence is padded as a sequence of 16 bytes after the string. It generates a file with `.encrypted` extension, which has the cryptographic payload prepended by IV. Which can be used for training the model.
 - `ml_pipeline/models/train_rf.py` is ingested with the 6 feature vector, to train RFC, which later is exported to `.pkl` file.
5. *src/kernel_ebpf*: has two files `vfs_monitor.c` and `ebpf_loader.py`.
- `vfs_monitor.c` contains the eBPF code which intercepts the Linux's `vfs_write` function and filters out non regular files to avoid oise form sockets/pipes, packages PID, timestamp, and Process name into a C struct and submits to `BPF_PERF_OUTPUT` map.
 - `ebpf_loader.py` is used to test the eBPF logic before deploying to the daemon.
6. *scripts/build_and_run.sh*: the shell script used to run the projects.

Some Headers and Structs in C++ file:

1. Critical Kernel & IPC Data structures: The implementation relies on two data-structures for high speed and deadlock-aware communications between kernel driver and user-space daemon.
 - struct sockaddr_un : used to assign an address to the Unix domain socket. This struct uses the local FS path for high speed communication between processes.
 - struct timeval: this provides microsecond scale precision in time intervals. This enforces the 500 ms bounded waiting time on IPC to guarantee Deadlock Recovery.
2. FUSE Architecture & Headers: FUSE is used to create a filesystem without modifying Linux Kernel Code. The OS communicates with FUSE via /dev/fuse.
 - <fuse.h> is the primary header providing the fuse API.
 - struct fuse_operations is a C structure provided by the fuse library, is used as a mapping structure, like, when we make a system calls they are mapped to our custom c++ functions, not the original functions.
 - OS uses metadata to render directories, so, `getattr` populates the struct stat, and `readdir` is used to populate the directory entries, which is later used by the commands like `ls` work correctly.
 - On the callback `open`, we open a file from physical storage and return an integer that tracks the file, is stored in `int fuse_file_info->fh`, the `fuse_file_info` stores info on an open file. So, while reading or writing, the file can be handled efficiently.
3. IPC using Unix Domain Sockets: We use AF_UNIX to communicate between FUSE and the python daemon, so the communication is rapid, than using TCP/IP. Because UNIX sockets bypass the entire network stack entirely, writing directly to the kernel buffer for increased performance.
 - <sys/socket.h> and <sys/un.h> provide the socket structures required for local IPC.
 - The FUSE driver executes `send(sock, buf, size, 0)` to push 64KB intercepted file fragment to python, then blocks on `recv()` waiting for the verdict.
4. Process management & signals: when a ransomware is detected, the driver returns the error using -EACCES. If only -EACCES is returned, the process will not care and persists. So,
 - Using `fuse.get_context()->pid` we get the PID of the process trying to write.
 - When a honeyfile is touched, the kernel executes `kill(calling_pid, SIGKILL)`, where SIGKILL aka signal-9, cannot be ignored, and the OS kills the process.
5. Extended Berkley Filter Packets and Kprobes: eBFP allows running the sandboxed programs directly within OS kernel without modifying kernel Source code or loading risky kernel modules.

- K probes: here, the `vfs_monitor.c`, using `vfs_write` will trap the input in the pipeline, and extracts the telemetry on the process and will send the data to the `ml_daemon`.
- `BPF_PERF_OUTPUT`: Because the kernel and user space memory are segregated, the `perf_buffer` / ring buffer and will extract the structs from the kernel space and passes it to the python code.

Flow of Execution:

Step 1: Install FUSE and the BCC

```
sudo apt update
sudo apt install -y fuse libfuse-dev bpfcc-tools linux-headers-$(uname -r)
pip3 install scikit-learn numpy joblib cryptography bcc
```

Step 2: Generate Ransomware Dataset:

```
python3 ml_pipeline/datasets/generate_real_ransomware_data.py
```

Step 3: Deploy Honeyfiles:

```
python3 ml_pipeline/models/train_rf.py
```

Step 4: Boot EDR and Mount Fuse:

```
cd scripts
sudo ./build_and_run.sh
```

```
(fuse_env) sankalp-kunthe@sankalp-kunthe-VirtualBox: ~/deadlock-aware-fuse/scripts$ fusermount -d /tmp/secure_mount
(fuse_env) sankalp-kunthe@sankalp-kunthe-VirtualBox:~/deadlock-aware-fuse/scripts$ ./build_and_run.sh
Building the Fuse System which is aware of deadlock....
[*] Compiling C++ FUSE Driver.
[*] Starting User-Mode ML Daemon.
    INITIALIZING UNIFIED EDR SECURITY DAEMON
[*] Master Daemon PID: 3998 (Whitelisted against Deadlocks)
[*] Mounting FUSE to /tmp/secure_mount...
[*] Starting Deadlock Aware FUSE. Whitelisting PID: 3998
System is now active.
To test: try writing normal file and an encrypted payload to /tmp/secure_mount
To shut down Type: fusermount -u /tmp/secure_mount && kill 3994
(fuse_env) sankalp-kunthe@sankalp-kunthe-VirtualBox:~/deadlock-aware-fuse/scripts$ [*] 6-Dimensional AI Brain Loaded Successfully.
[+] Thread 1: FUSE Mathematical ML Bridge Active.
[+] Thread 2: eBPF Deep-Kernel Monitor Active. Threshold: >100 writes/sec
```

Demonstration:

Test 1: The Benign Worker:

```
echo "This is a standard text file containing normal user data." > /tmp/secure_mount/normal_file.txt
```

```
cat /tmp/secure_mount/normal_file.txt
```

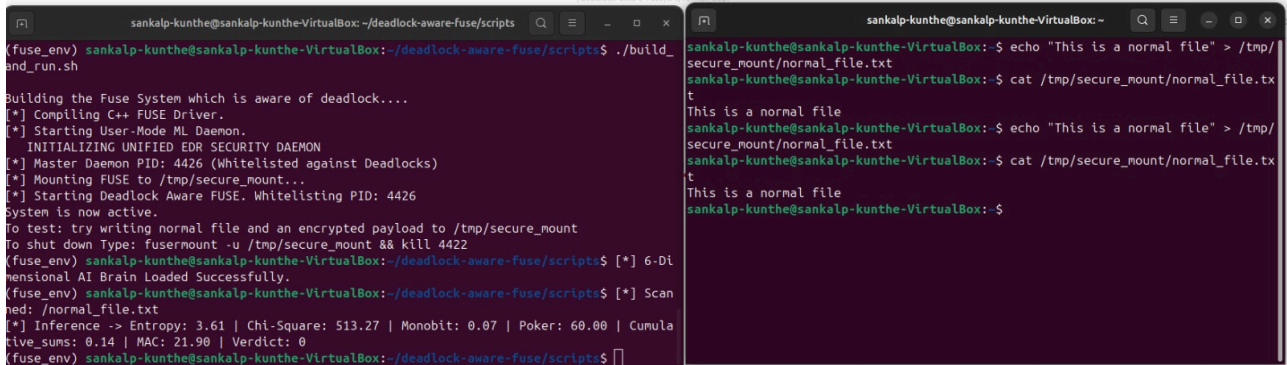
```
sankalp-kunthe@sankalp-kunthe-VirtualBox: ~/deadlock-aware-fuse/scripts
[*] Mounting FUSE to /tmp/secure_mount...
[*] Starting Deadlock Aware FUSE. Whitelisting PID: 4426
system is now active.
To test: try writing normal file and an encrypted payload to /tmp/secure_mount
To shut down Type: fusermount -u /tmp/secure_mount && kill 4422
(fuse_env) sankalp-kunthe@sankalp-kunthe-VirtualBox:~/deadlock-aware-fuse/scripts$ [*] 6-Dimensional AI Brain Loaded Successfully.
(fuse_env) sankalp-kunthe@sankalp-kunthe-VirtualBox:~/deadlock-aware-fuse/scripts$ [*] Scanned: /normal_file.txt
[*] Inference -> Entropy: 3.61 | Chi-Square: 513.27 | Monobit: 0.07 | Poker: 60.00 | Cumulative_sums: 0.14 | MAC: 21.90 | Verdict: 0
(fuse_env) sankalp-kunthe@sankalp-kunthe-VirtualBox:~/deadlock-aware-fuse/scripts$ [HONEYFILE TRIPWIRE] Rogue process touched '/passwords_2024.xlsx'!
MITIGATION] C++ Kernel Driver bypassed ML and executed SIGKILL.

[*] Scanned: /fake_malware.bin
[*] Inference -> Entropy: 8.00 | Chi-Square: 236.88 | Monobit: 0.00 | Poker: 11.86 | Cumulative_sums: 0.00 | MAC: 0.00 | Verdict: 1
Ransomware detected. Blocking write

sankalp-kunthe@sankalp-kunthe-VirtualBox:~$ echo "This is a normal file" > /tmp/secure_mount/normal_file.txt
sankalp-kunthe@sankalp-kunthe-VirtualBox:~$ cat /tmp/secure_mount/normal_file.txt
This is a normal file
sankalp-kunthe@sankalp-kunthe-VirtualBox:~$ echo "This is a normal file" > /tmp/secure_mount/normal_file.txt
sankalp-kunthe@sankalp-kunthe-VirtualBox:~$ cat /tmp/secure_mount/normal_file.txt
This is a normal file
sankalp-kunthe@sankalp-kunthe-VirtualBox:~$ bash -c 'echo "ransomware_encrypted_date" > /tmp/secure_mount/passwords_2024.xlsx'
Killed
sankalp-kunthe@sankalp-kunthe-VirtualBox:~$ dd if=/dev/urandom of=/tmp/secure_mount/fake_malware.bin bs=64K count=1
Killed
sankalp-kunthe@sankalp-kunthe-VirtualBox:~$
```

Test 2: The ML defense:

```
# Writing random bytes mimics AES-256 encryption density
bash -c 'head -c 65536 /dev/urandom > /tmp/secure_mount/
simulated_ransomware.encrypted'
```



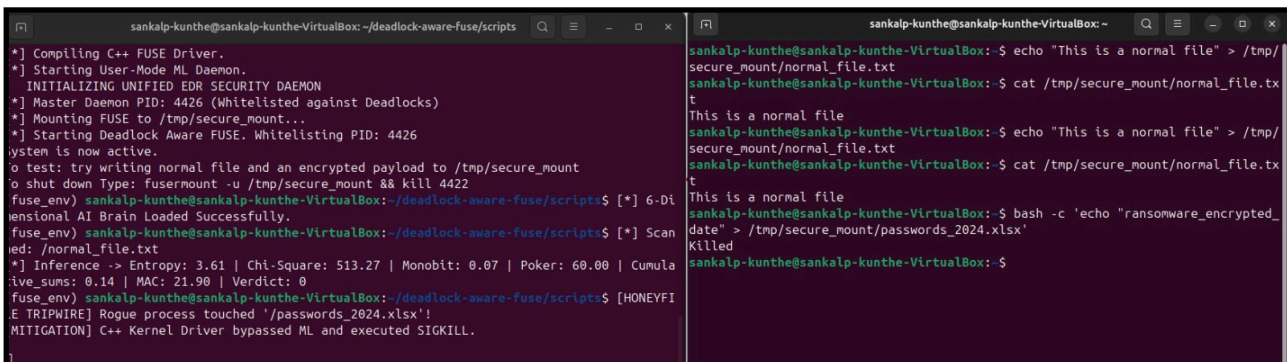
```
sankalp-kunthe@sankalp-kunthe-VirtualBox: ~/deadlock-aware-fuse/scripts
(fuse_env) sankalp-kunthe@sankalp-kunthe-VirtualBox:~/deadlock-aware-fuse/scripts$ ./build_and_run.sh

Building the Fuse System which is aware of deadlock....
[*] Compiling C++ FUSE Driver.
[*] Starting User-Mode ML Daemon.
    INITIALIZING UNIFIED EDR SECURITY DAEMON
[*] Master Daemon PID: 4426 (Whitelisted against Deadlocks)
[*] Mounting FUSE to /tmp/secure_mount...
[*] Starting Deadlock Aware FUSE. Whitelisting PID: 4426
System is now active.
To test: try writing normal file and an encrypted payload to /tmp/secure_mount
To shut down Type: fusermount -u /tmp/secure_mount && kill 4422
(fuse_env) sankalp-kunthe@sankalp-kunthe-VirtualBox:~/deadlock-aware-fuse/scripts$ [*] 6-Di
mensional AI Brain Loaded Successfully.
(fuse_env) sankalp-kunthe@sankalp-kunthe-VirtualBox:~/deadlock-aware-fuse/scripts$ [*] Scan
ned: /normal_file.txt
[*] Inference -> Entropy: 3.61 | Chi-Square: 513.27 | Monobit: 0.07 | Poker: 60.00 | Cumula
tive_sums: 0.14 | MAC: 21.90 | Verdict: 0
(fuse_env) sankalp-kunthe@sankalp-kunthe-VirtualBox:~/deadlock-aware-fuse/scripts$

sankalp-kunthe@sankalp-kunthe-VirtualBox: ~
sankalp-kunthe@sankalp-kunthe-VirtualBox:~$ echo "This is a normal file" > /tmp/
secure_mount/normal_file.txt
sankalp-kunthe@sankalp-kunthe-VirtualBox:~$ cat /tmp/secure_mount/normal_file.tx
t
This is a normal file
sankalp-kunthe@sankalp-kunthe-VirtualBox:~$ echo "This is a normal file" > /tmp/
secure_mount/normal_file.txt
sankalp-kunthe@sankalp-kunthe-VirtualBox:~$ cat /tmp/secure_mount/normal_file.tx
t
This is a normal file
sankalp-kunthe@sankalp-kunthe-VirtualBox:~$
```

Test 3: The Honeyfile Defense:

```
bash -c 'echo "encrypting your wallet" > /tmp/secure_mount/.wallet_backup.dat'
```



```
sankalp-kunthe@sankalp-kunthe-VirtualBox: ~/deadlock-aware-fuse/scripts
[*] Compiling C++ FUSE Driver.
[*] Starting User-Mode ML Daemon.
    INITIALIZING UNIFIED EDR SECURITY DAEMON
[*] Master Daemon PID: 4426 (Whitelisted against Deadlocks)
[*] Mounting FUSE to /tmp/secure_mount...
[*] Starting Deadlock Aware FUSE. Whitelisting PID: 4426
System is now active.
To test: try writing normal file and an encrypted payload to /tmp/secure_mount
To shut down Type: fusermount -u /tmp/secure_mount && kill 4422
(fuse_env) sankalp-kunthe@sankalp-kunthe-VirtualBox:~/deadlock-aware-fuse/scripts$ [*] 6-Di
mensional AI Brain Loaded Successfully.
(fuse_env) sankalp-kunthe@sankalp-kunthe-VirtualBox:~/deadlock-aware-fuse/scripts$ [*] Scan
ned: /normal_file.txt
[*] Inference -> Entropy: 3.61 | Chi-Square: 513.27 | Monobit: 0.07 | Poker: 60.00 | Cumula
tive_sums: 0.14 | MAC: 21.90 | Verdict: 0
(fuse_env) sankalp-kunthe@sankalp-kunthe-VirtualBox:~/deadlock-aware-fuse/scripts$ [HONEYFI
LE TRIPWIRE] Rogue process touched '/passwords_2024.xlsx'!
MITIGATION] C++ Kernel Driver bypassed ML and executed SIGKILL.

sankalp-kunthe@sankalp-kunthe-VirtualBox: ~
sankalp-kunthe@sankalp-kunthe-VirtualBox:~$ echo "This is a normal file" > /tmp/
secure_mount/normal_file.txt
sankalp-kunthe@sankalp-kunthe-VirtualBox:~$ cat /tmp/secure_mount/normal_file.tx
t
This is a normal file
sankalp-kunthe@sankalp-kunthe-VirtualBox:~$ echo "This is a normal file" > /tmp/
secure_mount/normal_file.txt
sankalp-kunthe@sankalp-kunthe-VirtualBox:~$ cat /tmp/secure_mount/normal_file.tx
t
This is a normal file
sankalp-kunthe@sankalp-kunthe-VirtualBox:~$ bash -c 'echo "ransomware_encrypted_
date" > /tmp/secure_mount/passwords_2024.xlsx'
Killed
sankalp-kunthe@sankalp-kunthe-VirtualBox:~$
```

Test 4: The eBPF behavioural Defense:

```
bash -c 'for i in {1..200}; do echo "encrypted" > /tmp/secure_mount/spam_$i.txt;
done'
```

References:

1. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). John Wiley & Sons.
2. Szeredi, M. (2005). *Filesystem in Userspace*. Linux Kernel Documentation.
3. Corbet, J. (2014). *BPF: the universal in-kernel virtual machine*. LWN.net.
4. National Institute of Standards and Technology (NIST). (2001). *Security Requirements for Cryptographic Modules* (FIPS PUB 140-2). U.S. Department of Commerce.